

Funktionsdokumentation — Vorpruefung

Stand 17.08.2026

Contents

vorpruefung/run.py	2
Modulkopf	2
Funktionen	2
vorpruefung/v0_aufteilung.py	2
Modulkopf	2
Kommentarbloecke (1)	3
Funktionen	3
vorpruefung/v1_baselines.py	4
Modulkopf	4
Funktionen	5
vorpruefung/v2_eignung.py	8
Modulkopf	8
Kommentarbloecke (5)	9
Funktionen	10
vorpruefung/v3_spezifikation.py	13
Modulkopf	13
Kommentarbloecke (1)	14
Funktionen	14
vorpruefung/v4_decke.py	15
Modulkopf	15
Kommentarbloecke (1)	17
Funktionen	17

vorpruefung/run.py

47 Zeilen · 2 Funktionen

Modulkopf

DER EINE BEFEHL der Vorpruefung.

```
python vorpruefung/run.py
```

Ablauf:

```
1 v1_baselines.py -> results/{regression,klassifikation}/baselines_*.csv
                        Stufe 1 (trivial) und Stufe 2 (einfachste passende Form)
2 v2_eignung.py   -> results/eignungspruefung/eignungspruefung.md
                        Welche Verfahrensklasse passt zu welcher Zielgroesse?
```

Reihenfolge ist zwingend: Die Eignungspruefung Liest die Baseline-Werte.

Danach steht fest, was die Vergleichsverfahren in modelle/ schlagen muessen und warum sie ueberhaupt antreten. Voraussetzung ist ein Lauf von prep/build.py.

Funktionen

schritt(nummer, titel)

Zeile 25 · 2 Zeilen

kein Docstring

main()

Zeile 29 · 15 Zeilen

kein Docstring

vorpruefung/v0_aufteilung.py

224 Zeilen · 4 Funktionen

Modulkopf

Wiederholte Splits - die eine Stelle, an der die Fold-Zuteilung je Wiederholung entsteht.

WOZU DIESE DATEI UEBERHAUPT EXISTIERT

Die Grundaufteilung steht als Spalten `fold` und `ist\_holdout` in beiden Parquet-Dateien; erzeugt hat sie `prep/s2\_datensaetze.ergaenze\_aufteilung()`. Fuer die WIEDERHOLTEN Splits (docs/04_MODELLIERUNG.md, Abschnitt 2) reicht sie nicht aus. Zwei nachgewiesene Gruende, beide am 05.08.2026 am Datensatz gemessen (docs/07_BEFUNDE.md, B-1 und B-2):

- 1 Der `versatz` verteilt die Stadtteile reihum auf $N_FOLDS + 1 = 6$ Gruppen. Wer i den Platz i belegt, landet in Gruppe $(i + versatz) \% 6$. Zwei Stadtteile liegen also genau dann in derselben Gruppe, wenn ihre Plaetze modulo 6 uebereinstimmen - UNABHAENGIG vom Versatz. Der Versatz rotiert damit nur die Beschriftung der Gruppen, nicht ihre Zusammensetzung. Ueber

versatz 0..9 entstehen 6 verschiedene Konstellationen, davon 4 Dubletten.

- 2 Rotiert die Beschriftung, rotiert auch Gruppe 0 - und Gruppe 0 IST das Hold-out. Gemessen: bei versatz = 1 liegt kein einziger der sechs urspruenglichen Hold-out-Stadtteile mehr im Hold-out. Die Wiederholungen 1 bis 9 wuerden auf genau den Stadtteilen trainieren und testen, die bis zur Schlussbewertung unberuehrt bleiben muessen.

Deshalb hier eine eigene Funktion mit drei Zusagen:

Das Hold-out bleibt FEST	die sechs Stadtteile mit <code>ist_holdout == 1</code> aus der Datei, in jeder Wiederholung dieselben
Die Stratifizierung bleibt	sortiert nach brand-dominierten Monaten, bei Gleichstand nach Bevoelkerung (Decision Log #30)
Wiederholung 0 = die Datei	bitgenau dieselbe fold-Spalte; das wird bei jedem Aufruf per assert nachgeprueft

WIE DIE WIEDERHOLUNGEN ENTSTEHEN

Die 29 Entwicklungsstadtteile werden wie bisher nach (selten, bev) absteigend sortiert und reihum ausgeteilt. Neu ist nur, dass vor dem Austeilen INNERHALB der Rangblocke gemischt wird - Block 0 sind die Plaetze 0-4, Block 1 die Plaetze 5-9 und so fort:

Rangblock 0	[Bayview, Bernal, Portola, Seacliff, Twin Peaks]
Rangblock 1	[...] -> jeder Block liefert genau
...	EINEN Stadtteil je Fold

Damit bekommt jeder Fold weiterhin genau einen Stadtteil aus jedem Rangblock - die doppelte Stratifizierung ueberlebt das Mischen unveraendert, und die Foldgroessen bleiben 6/6/6/6/5. Anders als beim Versatz aendert sich aber die ZUSAMMENSETZUNG der Folds, nicht nur ihre Nummer. Genau das brauchen die wiederholten Splits.

Kein Leakage: Gemischt wird ausschliesslich die Frage, welche Stadtteile gemeinsam getestet werden. Kein Modell sieht dadurch eine Zeile mehr.

Benutzt von `vorpruefung/v1\_baselines.py`, `modelle/m02\_menge.py` und `modelle/m03\_struktur.py` - alle drei muessen dieselbe Zuteilung sehen, sonst vergleicht der gepaarte Wilcoxon-Test still auf verschiedenen Zeilen.

Selbsttest:
python vorpruefung/v0_aufteilung.py

Kommentarblocke (1)

Zeile 166

=====
Selbsttest - beantwortet die vier Fragen, an denen diese Datei haengt
=====

Funktionen

`selten_je_stadtteil(klassifikation)`

Zeile 84 · 15 Zeilen

Zahl der brand-dominierten Monate je Stadtteil - das Stratifizierungsmass.

Wortgleich zu dem, was ``prep/s2_datensaetze.run()`` beim Bau der Dateien gerechnet hat. Es steht hier noch einmal, weil die Modellskripte den Wert brauchen, er aber nicht in den Dateien abgelegt ist: In ``regression.parquet`` gibt es keine Klassenspalte, und die fold-Spalte allein sagt nicht, WIE sie zustande kam.

Fuer die Regression heisst das: ``klassifikation.parquet`` mitlesen, auch wenn nur die Menge modelliert wird. Das ist kein Leakage - die Zahl geht in kein Modell ein, sie bestimmt nur, welche Stadtteile gemeinsam getestet werden.

wiederholte_aufteilung(daten, wiederholung, selten)

Zeile 101 · 52 Zeilen

Schreibt die fold-Spalte fuer eine Wiederholung. Hold-out bleibt fest.

``wiederholung`` 0 liefert exakt die Aufteilung aus der Datei - das wird per assert geprueft, nicht nur behauptet. 1 bis WIEDERHOLUNGEN-1 liefern verschiedene Zusammensetzungen bei gleicher Stratifizierung.

``selten`` ist die Reihe aus ``selten_je_stadtteil()``. Fehlt sie, wird nur nach Bevoelkerung stratifiziert - das reproduziert die Dateien NICHT und ist nur fuer Sonderfaelle gedacht.

Rueckgabe ist eine Kopie mit neu belegter Spalte ``fold``; ``ist_holdout`` wird unveraendert uebernommen. Die Hold-out-Zeilen behalten `fold = 0` und werden von ``fold_masken()`` in jeder Wiederholung ausgeschlossen.

entwicklung_und_holdout(daten)

Zeile 155 · 9 Zeilen

Masken fuer die Schlussbewertung: 29 Entwicklungs- gegen 6 Hold-out-Stadtteile.

Das Gegenstueck zu ``fold_masken()`` fuer den einen Lauf, der das Hold-out ueberhaupt anfassen darf. Steht hier und nicht im Modellskript, damit es genau eine Stelle im Repo gibt, an der ``ist_holdout == 1`` gelesen wird.

_selbsttest()

Zeile 169 · 52 Zeilen

kein Docstring

vorpruefung/v1_baselines.py

378 Zeilen · 8 Funktionen

Modulkopf

Stufe 1 und 2: die Messlatte.

Eine Baseline ist eine bewusst einfache Regel, die dieselbe Aufgabe loest und dieselben Daten sieht. Sie legt fest, was die Vergleichsverfahren mindestens schlagen muessen. Es gibt zwei Stufen:

STUFE 1 Triviale Referenz - benutzt KEIN einziges Merkmal.
Regression: Gesamtittelwert der Trainingsstadtteile
Klassifikation: immer die haeufigste Klasse

Beantwortet: Steckt in den Merkmalen ueberhaupt Information?

STUFE 2 Einfachste Referenz, die zur DATENFORM passt - benutzt alle Merkmale, aber in der simpelsten Form.
Regression: Poisson-GLM mit Offset (Zaehldaten mit Exposition)
Klassifikation: multinomiale logistische Regression (nominale Klassen)
Beide: kanonischer Link, unpenalisierte Maximum-Likelihood, KEIN freier Hyperparameter - deshalb kein Tuning (Decision Log #45).
Beantwortet: Wie weit kommt man mit der einfachen Form?

Stufe 3 sind die Vergleichsverfahren in modelle/. Ihre Aufgabe ist zu zeigen, dass sie Stufe 2 schlagen - sonst hat sich der Mehraufwand nicht gelohnt.

Alle Baselines laufen ueber denselben STADTTEIL-SPLIT wie die Modelle: Der Teststadtteil ist unbekannt. Das Hold-out bleibt unberuehrt.

WARUM UEBER ALLE 10 WIEDERHOLUNGEN (Ergaenzung 05.08.2026)

Bis dahin lief hier nur die Aufteilung, die als `fold`-Spalte in den Dateien steht - also funf Laeufe. Die Vergleichsverfahren erzeugen 50. Die Primaeraussage nach Decision Log #34 ist aber ein GEPAARTER Test „Verfahren gegen Stufe 2“, und der braucht je Lauf einen Gegenwert auf DENSELBEN Testzeilen. Fuer 45 der 50 Laeufe gab es keinen.

Die Baseline ist damit kein Referenzwert, sondern ein Mitbewerber unter identischem Protokoll - so verlangt es auch Schroeters Auflage C („fuer alle Vergleichsmodelle identische Merkmale und Splits“). Der Aufwand faellt nicht ins Gewicht: 50 GLM-Anpassungen kosten zusammen wenige Sekunden.

Eingang: data/processed/{regression,klassifikation}.parquet
Ausgang: results/regression/baselines_{folds,mittel}.csv
results/klassifikation/baselines_klasse.csv

Ausfuehren:
python vorpruefung/v1_baselines.py

Funktionen

bewerte_regression(y_true, y_pred)

Zeile 78 · 8 Zeilen

RMSE, MAE, R2 - immer auf der ORIGINALSKALA der Zielgroesse.

poisson_glm(train, test, merkmale)

Zeile 89 · 46 Zeilen

Stufe 2 der Regression: vorhergesagte Einsatzzahlen.

`merkmale` ist der Merkmalssatz; ohne Angabe der volle. Der Parameter existiert allein fuer die Faktorgruppen-Ablation in `m04\_shap.py`, die dasselbe Modell mit einer weggelassenen Gruppe anpasst. Ohne ihn muesste die Ablation die Spezifikation nachbauen - und dann gaebe es sie zweimal. Der Offset bleibt in jeder Variante bestehen: Er ist keine Merkmalsspalte.

Poisson-GLM mit kanonischem log-Link, per unpenalisierter Maximum-Likelihood angepasst. `log(Bevoelkerung)` geht als OFFSET ein, also mit fest auf 1 gesetztem Koeffizienten: Das Modell schaeetzt Einsaetze JE EINWOHNER und multipliziert am Ende hoch - sonst sagt es vor allem die

Stadtteilgroesse vorher (#13). Kein freier Hyperparameter, also kein Tuning.

WARUM POISSON UND NICHT NEGATIVE BINOMIAL (Decision Log #45). Die Zaehldaten sind ueberdispers (Dispersionsindex 62,8), die Poisson-Varianzannahme $\text{Var} = \mu$ ist also verletzt. Das ist folgenlos fuer den Zweck dieser Baseline: Der Poisson-Schaetzer bleibt konsistent, solange der BEDINGTE MITTELWERT richtig spezifiziert ist, unabhaengig von der Varianzstruktur (Gourieroux, Monfort & Trognon 1984, "Pseudo Maximum Likelihood Methods", Econometrica 52, 701-720). Was die Ueberdispersion beschaedigt, sind die STANDARDFEHLER - und die werden hier nicht verwendet, weil die Baseline ausschliesslich Punktvorhersagen liefert. Keine Koeffiziententests, keine Konfidenzintervalle.

Die Negative Binomial waere die Erweiterung fuer korrekte Inferenz. Sie loest ein Problem, das wir nicht haben, und ist damit nicht mehr "die einfachste Form, die zur Datenform passt".

Die Rate entsteht aus DERSELBEN Anpassung, geteilt durch die Bevoelkerung - ein zweites Modell waere eine zweite Spezifikation.

logit_glm(train, merkmale)

Zeile 137 · 41 Zeilen

Stufe 2 der Klassifikation: das angepasste multinomiale Logit.

`merkmale` wie bei `poisson\_glm()`: ohne Angabe der volle Satz, sonst der reduzierte fuer die Faktorguppen-Ablation.

DIE EINZIGE STELLE, AN DER DIESES MODELL SPEZIFIZIERT IST (10.08.2026). Bis dahin baute `m03\_struktur.hold\_out()` es ein zweites Mal nach - dieselben vier Argumente, an zwei Orten aufgeschrieben. Aendert jemand eines davon, misst die Kreuzvalidierung still gegen ein anderes Modell als die Schlussbewertung, und keine Pruefung schlaegt an: `pruefe\_zahlen.py` vergleicht Dokumentation gegen `results/`, nicht Code gegen Code.

Der Mengenstrang war immer richtig gebaut - `m02\_menge.hold\_out()` importiert `poisson\_glm` aus dieser Datei. Der Fehler war die Asymmetrie: derselbe Gedanke, einmal umgesetzt und einmal nicht.

Linear in den Log-Odds, unpenalisiert ($C = \text{inf}$; `penalty=None` ist seit scikit-learn 1.8 veraltet, die Schaetzung ist bitgleich). Kein freier Hyperparameter, also kein Tuning (#45). `class\_weight="balanced"` statt Resampling - kein SMOTE, keine duplizierte oder gelöschte Zeile.

RUECKGABE IST DAS MODELL, nicht die Vorhersage - anders als bei `poisson\_glm`. Beide Aufrufer brauchen aus DERSELBEN Anpassung drei Dinge: Klassenvorhersage, Wahrscheinlichkeiten und die Klassenreihenfolge des Modells. Ein zweites Fitten da fuer waere Verschwendung.

KONVERGENZWARNUNGEN werden hier bewusst NICHT abgefangen. Sie gehoeren dem Aufrufer: `klassifikation()` zaehlt sie und berichtet sie (docs/04_MODELLIERUNG.md, Sonderfaelle). Wurde diese Funktion sie schlucken, waere die Zahl still null.

regression(panel, selten)

Zeile 180 · 37 Zeilen

Beide Mengen-Zielgroessen, Stufe 1 und 2, je Wiederholung und Fold.

Die Rate ergibt sich aus derselben Poisson-Vorhersage geteilt durch die Bevoelkerung - ein zweites Modell waere eine zweite Spezifikation und damit unfair gegenueber den Vergleichsverfahren.

50 Laeufe (10 Wiederholungen x 5 Folds) x 2 Zielgroessen x 2 Modelle
(Nullmarke, Poisson-GLM) = 200 Zeilen.

`_zweistufig(df, schluessel, masse)`

Zeile 219 · 28 Zeilen

Zweistufige Aggregation ueber alle Laeufe des Durchgangs.

ZWEISTUFIG heisst: erst je Wiederholung ueber die 5 Folds mitteln, dann die Streuung DIESER Werte berichten.

``std_folds`` ueber alle 50 Einzellaeufe. Zu optimistisch, weil die Laeufe nicht unabh angig sind - es sind dieselben 29 Stadtteile in zehn Gruppierungen.

``std_wiederholungen`` ueber die 10 Wiederholungsmittel. MASSGEBLICH (docs/06_RISIKEN.md, R-5).

Eine Datei beschreibt genau einen Durchgang. Frueher fuehrte sie zusaetzlich Zeilen fuer Wiederholung 0 allein - das war historischer Ballast und vor allem eine Falle: Wer den Filter vergisst, bekommt stillschweigend die falsche Baseline. Wer diese Werte braucht, filtert ``baselines_folds.csv`` auf ``wiederholung == 0``; dort steht jeder Einzellauf.

`klassifikation(kl, selten)`

Zeile 250 · 67 Zeilen

Beide Stufen der Klassifikation, je Wiederholung und Fold.

STUFE 1, Mehrheitsklasse: sagt immer die im Training haeufigste Einsatzart vorher. Accuracy faellt hoch aus, Macro-F1 niedrig - genau deshalb ist Macro-F1 das massgebliche Guetemass.

STUFE 2, multinomiale logistische Regression: das Gegenstueck zum Poisson-GLM - dieselbe Modellklasse, derselbe kanonische Link, derselbe Verzicht auf einen Strafterm. Sie ist die einfachste Form, die zu einer nominalen Zielgroesse passt. RF und XGBoost muessen SIE schlagen, nicht die Mehrheitsklasse (Decision Log #33). Spezifiziert ist sie in ``logit_glm()`` - an genau einer Stelle, siehe dort.

Zwei Ergaenzungen vom 05.08.2026, beide additiv:

- Schleife ueber die 10 Wiederholungen, damit m03 gepaart testen kann
- Macro-AUROC wird fuer Stufe 2 mitgerechnet. Ohne sie gaebe es fuer das zweite Guetemass der Klassifikation keine Messlatte. Fuer die Mehrheitsklasse ist sie nicht definiert (eine konstante Vorhersage hat keine Rangfolge) und bleibt leer - NICHT 0,5, das waere eine erfundene Zahl.

Konvergenzwarnungen werden GEZAEHLT und zurueckgegeben, nicht unterdrueckt (docs/04_MODELLIERUNG.md, Sonderfalle).

`_macro_auroc(y_true, proba, klassen_modell, klassen_alle)`

Zeile 319 · 18 Zeilen

Macro-AUROC (One-vs-Rest), oder NaN wenn im Testfold eine Klasse fehlt.

NICHT durch 0,5 oder 0 ersetzen: Ein erfundener Wert zoege den Mittelwert nach unten und saeue wie ein Messergebnis aus (docs/04_MODELLIERUNG.md, Sonderfaelle). Durch die doppelte Stratifizierung (#30) sollte der Fall nicht eintreten - wenn doch, muss er sichtbar bleiben.

`run()`

Zeile 340 · 35 Zeilen

kein Docstring

vorpruefung/v2_eignung.py

699 Zeilen · 12 Funktionen

Modulkopf

Eignungspruefung: Passen die gewaehlten Verfahren zu den Zielgroessen?

Zweiter Schritt der Vorpruefung. Setzt `v1\_baselines.py` voraus - die Baseline-Werte werden gelesen, nicht neu gerechnet.

Sechs Belege, mehr nicht:

- | | | | |
|---|--------------------------------------|----|--|
| 1 | Zaehldaten sind ueberdispers | -> | zaehldatengerechte Verlustfunktionen (#42); die Stufe-2-Baseline bleibt das Poisson-GLM (#45, Begrueendung in Abschnitt 1) |
| 2 | Zusammenhaenge sind nicht linear | -> | Ridge auf $\log(1+y)$, nicht roh |
| 3 | Lineare Spezifikation reicht nicht | -> | Random Forest und XGBoost |
| 4 | Teststadtteile liegen oft ausserhalb | -> | Limitation, keine Verfahrensfrage |
| 5 | Merkmale trennen auch die Einsatzart | -> | RF und XGBoost im 2. Strang |
| 6 | Anforderungen je Verfahren geprueft | -> | Tabelle mit Teststatistik und p-Wert, Auflage vom 10.08.2026 |

Abschnitt 6 ist Auflage Schroeter (10.08.2026): "Pruefung ob die Algorithmen auf den Daten passen, z.B. Varianzgleichheit, linearer Zusammenhang ... Jeder Algorithmus sollte dargestellt werden ... Test laufen lassen: in Tabelle Statistiken mit p-Werten anzeigen." Die Abschnitte 1 bis 5 belegen die VERFAHRENSWAHL, Abschnitt 6 fuehrt die Anforderungen je Verfahren zusammen - einschliesslich der Zeilen, in denen eine Anforderung GAR NICHT besteht. Genau die gehoeren hin: Dass Baumverfahren keine Verteilungsannahme haben, ist eine Aussage und keine Auslassung.

Abschnitt 2 ist Auflage Schroeter (R7): "erstmal plotten, falls keine lineare Baseline, KEIN lineares Regressionsmodell." Deshalb Streudiagramme und Residuenanalyse, beides als Abbildung.

Abschnitt 5 ist noetig, weil die Regression den Klassifikationsstrang NICHT mitbeantwortet: Dass der Zusammenhang zur Anzahl gekrueumt ist, sagt nichts darueber, ob dieselben Merkmale die Art der Einsaetze trennen koennen.

Was diese Pruefung NICHT leistet: Sie unterscheidet nicht zwischen Random

Forest und XGBoost. Welche der beiden Strategien gewinnt, ist die empirische Forschungsfrage der Arbeit - vorab noetig ist nur, dass beide plausibel sind.

Gerechnet wird ausschliesslich auf den TRAININGSSTADTTEILEN VON FOLD 1 - die Teststadtteile duerfen keine Modellentscheidung beeinflussen. Ausgenommen sind Abschnitt 4 (Extrapolation, betrifft alle Folds naturgemaess) und die aus vl_baselines.py gelesenen Referenzwerte.

Der Bericht ist ein BEFUNDBLATT, keine Kapitelvorlage: Er liefert Zahlen und Abbildungen, die Argumentation fuer Kapitel 6.2 wird von Hand geschrieben.

Eingang: data/processed/{regression,klassifikation}.parquet
results/klassifikation/baselines_klasse.csv
Ausgang: results/eignungspruefung/eignungspruefung.md + 2 Abbildungen
results/eignungspruefung/annahmen.csv Abschnitt 6, maschinenlesbar
results/eignungspruefung/qq_residuen.csv Rohdaten fuer Abbildung A10

Ausfuehren:
python vorpruefung/v2_eignung.py

Kommentarbloecke (5)

Zeile 186

Massgeblich ist das MAXIMUM beider Korrelationen, nicht Pearson allein.
Grund: Ist ein Zusammenhang stark gekruemmt, faellt Pearson gerade deshalb ab - ein reiner Pearson-Filter wuerde also ausgerechnet den staerksten Kruemmungsbefund aussortieren. Umgekehrt bleibt der Zweck erhalten: Liegen BEIDE Korrelationen nahe null, ist ihr Abstand Rauschen und sagt nichts ueber die Funktionsform.

Zeile 387

DIE SPALTE "je Fold" ZEIGT NUR WIEDERHOLUNG 0 - nachgezogen 10.08.2026.
'vl_baselines.py' liefert seit dem 05.08. 50 Laeufe statt 5 (10 Wiederholungen x 5 Folds), damit m03 gepaart testen kann. Diese Tabelle hat weiterhin alle Zeilen aufgereiht: eine Zelle mit 50 durch Punkte getrennten Werten, unlesbar und als "je Fold" auch falsch beschriftet.

Der MITTELWERT bleibt bewusst ueber ALLE Laeufe gebildet - das ist der Wert, der in 03_STAND.md steht und gegen den m03 antritt. Gezeigt werden die fuefn Folds der Wiederholung 0, weil sie die Aufteilung aus der Datei sind (v0_aufteilung) und damit die nachvollziehbare.

Zeile 494

--- Ueberdispersion, formal (Cameron & Trivedi 1990) -----
Hilfsregression: $z = ((y - \mu)^2 - y) / \mu$ auf μ , ohne Konstante. Der Koeffizient ist der Dispersionsparameter α der NB2-Form, H_0 lautet $\alpha = 0$ (Equidispersion). Einseitig, weil Unterdispersion hier keine sinnvolle Gegenhypothese waere.

Zeile 506

--- Streuung und Verteilung der Residuen -----
Geprueft wird das lineare Modell, fuer das Ridge steht: OLS auf $\log(1+y)$ mit denselben zwölf Merkmalen. Ridge selbst hat denselben Erwartungswert und unterscheidet sich nur durch den Strafterm.

Zeile 668

Die Abschnitte 1, 3 und 4 geben ihre Kennzahlen zurueck, damit Abschnitt 6 sie nicht ein zweites Mal rechnen muss. Zweimal gerechnet

hiesse zwei Zahlen, die auseinanderlaufen koennen.

Funktionen

log(txt)

Zeile 81 · 3 Zeilen

kein Docstring

speichere(fig, name)

Zeile 86 · 5 Zeilen

kein Docstring

dispersion(train)

Zeile 94 · 68 Zeilen

Beleg 1: Die Zaehldaten sind ueberdispers - und was daraus folgt.

NEU GEFASST AM 10.08.2026. Bis dahin schloss dieser Abschnitt aus der Overdispersion, Poisson scheide aus und die Negative Binomial sei die passende Baseline. Decision Log #45 hat am 06.08. das Gegenteil entschieden und ist am 08.08. freigegeben worden - der Abschnitt argumentierte danach gegen die eigene Umsetzung, und die erzeugte `eignungspruefung.md` trug den Widerspruch weiter. Kein Rechenfehler, sondern Drift.

DIE KORREKTE FOLGERUNG hat zwei Aeste, und nur der erste betrifft die Baseline:

Verlustfunktion Ein quadratischer Fehler auf rohen Zaehldaten ist bei diesem Dispersionsindex unangemessen. Daraus folgt `reg:tweedie` fuer XGBoost und `criterion="poisson"` fuer den Random Forest (#42). Das ist die eigentliche Konsequenz aus dieser Messung.

Baseline Die Overdispersion verletzt die Poisson-Varianzannahme $Var = \mu$. Beschadigt werden dadurch die STANDARDFEHLER, nicht die Konsistenz des geschaetzten bedingten Mittelwerts (Gourieroux, Monfort & Trognon 1984). Eine Baseline, die ausschliesslich Punktvorhersagen liefert - keine Koeffiziententests, keine Konfidenzintervalle -, ist davon nicht betroffen. Das Poisson-GLM bleibt Stufe 2 (#45).

Die Negative Binomial waere die Erweiterung fuer korrekte INFERENZ. Sie loest damit ein Problem, das diese Baseline nicht hat, und bringt mit dem Dispersionsparameter eine zusaetzliche Groesse mit - sie ist dann nicht mehr "die einfachste Form, die zur Datenform passt".

Der gemessene Index bleibt unveraendert und wird weiterhin berichtet. Er ist nicht falsch geworden, er traegt nur eine andere Schlussfolgerung.

linearitaet(train)

Zeile 165 · 94 Zeilen

Beleg 2: Auflage Schroeter (R7) - plotten, bevor Ridge eingesetzt wird.

Zwei Diagnosen. Pearson misst den LINEAREN, Spearman den MONOTONEN Zusammenhang; klaffen sie auseinander, ist der Zusammenhang gekrümmt. Bewertet wird das nur, wo die Korrelation überhaupt substantiell ist - bei einer Korrelation nahe null ist der Abstand Rauschen.

Danach Ridge einmal auf der Rohskala und einmal auf $\log(1+y)$, mit Residuenbild. Ein Trichter zeigt, dass der Fehler mit dem Niveau wächst.

spezifikation(train)

Zeile 262 · 45 Zeilen

Beleg 3: Der RESET-Test verwirft die lineare Spezifikation.

Er prüft, ob Potenzen der Vorhersage noch etwas erklären. Tun sie das, hat das lineare Modell Struktur übrig gelassen. Die Interaktionsterme zeigen anschliessend, dass ein Teil davon Wechselwirkungen sind.

extrapolation(panel)

Zeile 310 · 25 Zeilen

Beleg 4: Wie oft liegt ein Teststadtteil ausserhalb des Gelernten?

klassifikation(kl)

Zeile 338 · 84 Zeilen

Beleg 5: Taugen dieselben Merkmale auch fuer die Einsatzart?

Eine Frage, die die Regression NICHT mitbeantwortet. Dass der Zusammenhang zur ANZAHL gekrümmt ist, sagt nichts darueber, ob dieselben Merkmale die ART trennen koennen - das sind zwei verschiedene Fragen an dieselben Spalten.

Geprüft wird per Kruskal-Wallis je Merkmal ueber die vier Klassen: nichtparametrisch, vertraegt ungleich grosse Gruppen. Trennt kein Merkmal, ist die Zielgrosse mit diesen Praediktoren nicht vorhersagbar - und zwar fuer JEDES Verfahren.

Danach werden die beiden Baseline-Stufen aus `v1_baselines.py` berichtet, um das Signal ins Verhaeltnis zu setzen: Wie viel davon schoepft ein lineares Modell aus? OB flexiblere Verfahren mehr herausholen, beantwortet `m03` - mit getunten Modellen ueber alle Wiederholungen und nicht mit einer Vorschau.

_z(wert, stellen)

Zeile 425 · 3 Zeilen

Teststatistik mit deutschem Dezimalkomma - wie die p-Spalte daneben.

_p(wert)

Zeile 430 · 12 Zeilen

p-Wert deutsch. Unter 0,001 wird nicht mehr beziffert, sondern begrenzt.

Grund: ``4.0e-47`` ist keine Information, die jemand liest - die Aussage ist "praktisch null". Bei $n = 3.036$ findet ein Test ohnehin fast jede Abweichung; die Effektgrosse traegt, nicht die Nachkommastelle.

annahmen(train, befunde)

Zeile 444 · 205 Zeilen

Beleg 6: Was verlangt jedes Verfahren - und haelt der Datensatz das?

AUFLAGE SCHROETER, 10.08.2026. Verlangt sind drei Dinge: die Anforderungen JE VERFAHREN dargestellt, formale Tests statt Augenmass, und beides in einer Tabelle mit Teststatistik und p-Wert.

DREI SORTEN VON ZEILEN, und die dritte ist die wichtigste:

erfuellt	die Anforderung besteht und ist eingehalten
verletzt	sie besteht und ist verletzt - dann steht in der Spalte "Konsequenz", was daraus folgt
nicht erforderlich	das Verfahren stellt diese Anforderung gar nicht

Die dritte Sorte wegzulassen waere der Fehler. Dass Random Forest keine Verteilungsannahme hat, ist eine AUSSAGE ueber das Verfahren - und sie ist der halbe Grund, warum es im Vergleich steht. Eine Tabelle, die nur verletzte Annahmen zeigt, laesst die Baumverfahren voraussetzungslos aussehen; eine, die sie ganz weglaesst, beantwortet die Auflage nicht.

DREI NEUE TESTS, die es vorher nicht gab:

Cameron & Trivedi (1990)	Hilfsregression auf Ueberdispersion. Der Dispersionsindex aus Abschnitt 1 ist eine Kennzahl, kein Test - hier steht der t-Wert.
Breusch-Pagan	Varianzgleichheit der Residuen. Woertlich in der Auflage genannt.
Jarque-Bera	Normalitaet der Residuen, dazu Schiefe und Woelbung. Die zugehoerige Abbildung ist A10; die Rohdaten dafuer schreibt diese Funktion nach `qq_residuen.csv`, gezeichnet wird in m05 - dieses Skript erzeugt Befunde, keine druckfertigen Abbildungen.

WAS HIER NICHT STEHT: die Multikollinearitaet. Der VIF wird in `m04\_shap.vif()` gerechnet, weil seine einzige echte Konsequenz die Interpretation der Beitraege betrifft. Ihn hier zu wiederholen hiesse, dieselbe Zahl an zwei Orten zu fuehren - genau die Fehlerquelle, die `tools/pruefe\_zahlen.py` bewacht.

innere Funktion z()

Eine Zeile der Anforderungstabelle.

`statistik` ist die LESBARE Fassung mit Dezimalkomma, `wert` dieselbe Zahl maschinenlesbar. Beides, weil `tools/pruefe\_zahlen.py` den Sollwert aus dieser Datei zieht und "t = 17,2" dafuer erst geparkt werden muesste - eine Zeichenkette, die man parst, ist eine Zeichenkette, die sich beim naechsten Formatwechsel anders parst.

main()

Zeile 652 · 44 Zeilen

kein Docstring

vorpruefung/v3_spezifikation.py

264 Zeilen · 6 Funktionen

Modulkopf

Haelte die diagnostizierte Nichtlinearitaet out-of-sample nach?

```
python vorpruefung/v3_spezifikation.py
```

Eingang: data/processed/regression.parquet

Ausgang: results/spezifikation/spezifikation_{folds,mittel}.csv

STAND: vollstaendig, 07.08.2026.

WOZU DIESES SKRIPT

`v2\_eignung.py` weist nach, dass die lineare Spezifikation nicht ausreicht: der RESET-Test verwirft sie deutlich ($F = 215,2$ bei Potenzen bis 2), und 45 Interaktionsterme heben das adjustierte R^2 von 0,805 auf 0,919. Daraus wurde die Wahl der Baumverfahren begruendet - sie fangen Kruemmung und Wechselwirkungen ohne Zutun ab.

Beide Kennzahlen sind IN-SAMPLE-Groessen, berechnet auf 3.828 Zeilen, die als unabhaengig behandelt werden. Tatsaechlich liegen 29 unabhaengige Stadtteile mit je 132 Monaten vor. Ein F-Test mit $n = 3.828$ findet praktisch jede Abweichung signifikant, und adjustiertes R^2 korrigiert fuer die Zahl der Parameter, nicht fuer die geklumpete Struktur.

Die Diagnose beantwortet also: STECKT in diesen Daten Struktur jenseits der Geraden? Der Verfahrensvergleich beantwortet eine andere Frage: UEBERTRAEGET sich diese Struktur auf unbekannte Stadtteile? Dieses Skript stellt genau diese zweite Frage - mit demselben Modell, demselben Split und denselben 50 Laeufen wie die Baseline, nur mit erweiterter Merkmalsmatrix.

Es ist damit kein Modellvorschlag. Keine der drei Erweiterungen tritt im Verfahrensvergleich an; sie dienen ausschliesslich der Interpretation des Hauptbefundes (docs/07_BEFUNDE.md, B-41).

DIE VIER SPEZIFIKATIONEN

Grundlage ist immer das Stufe-2-Modell aus `v1\_baselines.py`: Poisson-GLM mit log-Link und `log(Bevoelkerung)` als Offset, unpenalisiert angepasst.

linear	12 Terme	die 10 Praediktoren + monat_sin + monat_cos
quadrate	22 Terme	zusaetzlich die Quadrate der 10 Praediktoren
interaktionen	57 Terme	zusaetzlich alle 45 Paarprodukte der Praediktoren
beides	67 Terme	Quadrate und Paarprodukte

Die Saisonterme werden WEDER quadriert NOCH gekreuzt. $\text{monat_sin}^2 + \text{monat_cos}^2 = 1$ ist exakt kollinear mit der Konstanten; das Modell waere nicht identifiziert. Die 45 Paarprodukte entsprechen genau den 45 Interaktionstermen, die `v2\_eignung.py` bewertet - deshalb ist der Vergleich derselbe.

STANDARDISIERUNG. Die Merkmale werden je Fold auf den TRAININGSDATEN z-standardisiert, bevor Quadrate und Produkte gebildet werden. Das ist keine Modellentscheidung, sondern numerische Notwendigkeit: Das Quadrat des Medianeinkommens liegt in der Groessenordnung $1e10$, das Produkt zweier

Praediktoren bei $1e9$, und die IRLS-Iteration bricht auf dieser Konditionierung zusammen. Mathematisch ist die Standardisierung folgenlos - der aufgespannte Raum von $\{1, x, x^2\}$ ist derselbe wie der von $\{1, z, z^2\}$, die Vorhersagen sind identisch. Die Kennzahlen der Spalte `linear` muessen deshalb exakt die Stufe-2-Baseline aus `results/regression/baselines\_mittel.csv` reproduzieren; das prueft `\_selbsttest()` und bricht sonst ab.

KONVERGENZ. Mit 67 Termen auf 3.036 Trainingszeilen konvergiert die IRLS-Iteration nicht in jedem Fold. Nicht konvergierte Anpassungen werden GEZAEHLT und mitberichtet, nicht stillschweigend uebergangen und nicht entfernt - eine nicht konvergierte Anpassung ist Teil des Befundes, dass diese Spezifikation zu den Daten nicht passt.

PRUEFAUFTRAEGE

- Reproduziert `linear` die Stufe-2-Baseline auf drei Nachkommastellen?
Wenn nein: Abbruch, dann ist die Merkmalsmatrix nicht dieselbe.
- Wie viele der 200 Anpassungen sind nicht konvergiert, und in welchen Spezifikationen? Die Zahl gehoert in den Text.
- Ist der Abstand `linear` zu `interaktionen` groesser als der Abstand `linear` zu Random Forest? Nur dann traegt die Aussage "die Spezifikation bewegt mehr als die Verfahrenswahl" (docs/07_BEFUNDE.md, B-41).
- Das Hold-out bleibt unberuehrt: keine Zeile mit `ist_holdout == 1`.

Kommentarbloecke (1)

Zeile 169

Eine divergierte Anpassung kann beliebig grosse Vorhersagen liefern; die Kennzahlen werden dann unbrauchbar gross. Das ist so gewollt und wird nicht abgeschnitten - es IST das Ergebnis.

Funktionen

entwerfe(train, test, spezifikation)

Zeile 111 · 37 Zeilen

Merkmalsmatrizen fuer Training und Test, auf Trainingsdaten zentriert.

Gibt (X_train, X_test, Namen) zurueck, jeweils MIT Konstante an Position 0. Mittelwert und Streuung stammen ausschliesslich aus dem Training - der Teststadtteil darf die Transformation nicht mitbestimmen.

ein_lauf(train, test, spezifikation)

Zeile 150 · 25 Zeilen

Eine Poisson-Anpassung, eine Bewertung auf der Originalskala.

alle_laeufe(panel, selten)

Zeile 177 · 13 Zeilen

10 Wiederholungen x 5 Folds x 4 Spezifikationen = 200 Anpassungen.

zweistufig(df)

Zeile 192 · 15 Zeilen

Erst je Wiederholung ueber die Folds, dann ueber die Wiederholungen.

Dieselbe Regel wie ueberall sonst (docs/06_RISIKEN.md, R-5): massgeblich ist die Streuung der 10 Wiederholungsmittel, nicht die der 50 Einzellaefue.

_selbsttest(mittel)

Zeile 209 · 21 Zeilen

Die lineare Spezifikation MUSS die Stufe-2-Baseline reproduzieren.

Wenn nicht, sieht das Skript andere Merkmale oder andere Folds als `v1\_baselines.py` - dann ist jeder Vergleich in dieser Datei wertlos.

run()

Zeile 232 · 29 Zeilen

kein Docstring

vorpruefung/v4_decke.py

313 Zeilen · 7 Funktionen

Modulkopf

Wie gut KANN die Einsatzart bei dieser Zielgroesse ueberhaupt vorhergesagt werden?

python vorpruefung/v4_decke.py Entwicklungspanel, 29 Stadtteile
python vorpruefung/v4_decke.py holdout zusaetzlich die 6 gesperrten

Eingang: data/processed/klassifikation.parquet
 results/klassifikation/struktur_mittel.csv (optional, fuer die Quoten)
Ausgang: results/klassifikation/decke.csv
 results/klassifikation/decke_marge.csv
 results/klassifikation/decke_ausschoepfung.csv
 results/klassifikation/decke.md
 mit Argument "holdout" dieselben Dateien mit Endung _holdout

STAND: vollstaendig, 17.08.2026.

WOZU DIESES SKRIPT

Der Strukturstrang erreicht Macro-F1 um 0,33. Gegen die 1,0 einer fehlerfreien Vorhersage gehalten sieht das nach einem misslungenen Modell aus. Diese Lesart ist falsch, und dieses Skript belegt, warum: Sie vergleicht das Ergebnis mit einer Obergrenze, die bei DIESER Zielgroesse und DIESEM Merkmalssatz gar nicht erreichbar ist.

Zwei Obergrenzen begrenzen den Strukturstrang, und beide entstehen VOR jeder Modellwahl - die eine in der Konstruktion der Zielgroesse, die andere in der Struktur der Merkmale. Sie zu beziffern ist keine nachtraegliche Entlastung, sondern die Voraussetzung dafuer, Macro-F1 0,33 ueberhaupt einordnen zu koennen. Ohne sie bleibt jede Aussage ueber den Strukturstrang eine Vermutung.

DECKE A - LABEL-RAUSCHEN AUS DEM ARGMAX

`dominante\_einsatzart` ist kein beobachtetes Merkmal, sondern der argmax ueber vier Anteilsspalten desselben Stadtteil-Monats. Wo zwei Anteile dicht beieinander liegen, entscheidet der Zufall der Monatsziehung, welche Klasse gewinnt - nicht die Struktur des Stadtteils. Gemessen wird das mit einem parametrischen Bootstrap: Jeder Stadtteil-Monat wird aus Multinomial(N, p_beobachtet) neu gezogen und geprueft, ob der argmax kippt. Der Macro-F1 zwischen dem beobachteten und dem neu gezogenen Label ist die Guete, die ein Modell erreichte, das die wahren Klassenwahrscheinlichkeiten EXAKT kennt. Kein Verfahren kann darueber hinaus.

DECKE B - GRENZE DES STADTTEILWISSENS

Alle zwoelf Praediktoren sind stadtteilgebunden: die baulichen konstant ueber den gesamten Zeitraum, die sozialen konstant je Stadtteil-Jahr, der Kriminalitaetsindex zu 90 Prozent zwischen den Stadtteilen. Ein Modell kann aus ihnen nur Stadtteilwissen ziehen. Die zugehoerige Obergrenze ist deshalb die Guete einer Vorhersage, die jedem Stadtteil-Monat die Modalklasse SEINES Stadtteils zuweist - mehr traegt Stadtteilwissen nicht.

Diese Decke liegt DEUTLICH unter Decke A. Der Grund steht in der Tabelle `decke.csv`: Die Modalklassen der Stadtteile sind fast alle dieselbe.

AUSSCHOEPFUNG

Berichtet wird die baselinekorrigierte Quote

$$(\text{Modell} - \text{Mehrheitsklasse}) / (\text{Decke} - \text{Mehrheitsklasse})$$

Der Rohquotient Modell/Decke waere geschoent: Ein Modell, das nur die Mehrheitsklasse nachbaut, erreicht bereits Macro-F1 0,22 - dieser Sockel gehoert nicht zur Leistung des Modells und darf nicht mitgezaehlt werden.

FALLSTRICKE

- 1 DAS HOLD-OUT BLEIBT GESPERRT. Ohne das Argument "holdout" wird auf `ist\_holdout == 0` gefiltert, wie in m02 und m03. Die Decke ist zwar eine Eigenschaft der ZIELGROESSE und beruehrt keine Praediktoren - aber die Sperre gilt konstruktiv fuer alle Skripte, nicht nach Ermessen.
- 2 MONATE OHNE EINSATZE gibt es in dieser Tabelle nicht (N >= 1 in allen Zeilen), das Skript prueft es trotzdem. Bei N = 0 waere p undefiniert und `rng.multinomial` wuerde stumm einen Nullvektor liefern, dessen argmax immer auf die erste Klasse zeigt - eine erfundene Beobachtung.
- 3 DER BOOTSTRAP BRAUCHT EINEN FESTEN RANDOM_STATE. Ohne ihn schwankt Decke A zwischen zwei Laufen, und die Zahl in der Arbeit passt nicht mehr zur Zahl in der CSV. RANDOM_STATE steht in config_modelle.py und ist derselbe wie im Verfahrensvergleich.
- 4 DECKE A IST EINE OBERGRENZE, KEIN ZIELWERT. Sie beziffert, was bei perfekter Kenntnis der Klassenwahrscheinlichkeiten uebrig bliebe. Dass ein Modell sie nicht erreicht, ist kein Mangel - Decke B ist die bindende.

PRUEFAUFTRAEGE

- Liegt Decke B UNTER Decke A? Wenn nicht, ist etwas falsch: Stadtteilwissen kann das Label-Rauschen nicht unterbieten.
- Liegen beide Decken UEBER dem Macro-F1 der Mehrheitsklasse? Eine Decke unterhalb der trivialen Baseline waere ein Rechenfehler.

- Wie viele Stadtteile teilen dieselbe Modalklasse? Je hoeher diese Zahl, desto enger Decke B - das ist die inhaltliche Begrueundung des Befundes.
- Passt die Zeilenzahl? 4 Zeilen in decke.csv, 6 in decke_marge.csv.

Kommentarbloecke (1)

Zeile 290

Der Hold-out-Lauf schreibt in EIGENE Dateien. Sonst ueberschriebe die Schlussbewertung die Zahlen des Entwicklungspanels, auf die sich Kapitel 7.2 bezieht - derselbe Fehler, den m02 und m03 mit einer getrennten holdout.csv vermeiden.

Funktionen

`_macro_f1(a, b)`

Zeile 120 · 2 Zeilen

kein Docstring

`decke_a(panel)`

Zeile 124 · 25 Zeilen

Label-Rauschen des argmax, parametrischer Bootstrap.

Jeder Stadtteil-Monat wird aus Multinomial(N, p_beobachtet) neu gezogen. Zurueck kommen der mittlere Macro-F1 zwischen beobachtetem und neu gezogenem Label, dessen Streuung ueber die Ziehungen und der Anteil der Zeilen, deren argmax dabei mindestens einmal kippt.

FALLSTRICK 2: Zeilen mit N = 0 werden vorher ausgeschlossen, nicht auf eine Klasse gesetzt.

`decke_b(panel)`

Zeile 151 · 12 Zeilen

Obergrenze des Stadtteilwissens: Modalklasse je Stadtteil.

Mehr als die haeufigste Klasse seines Stadtteils kann ein Modell aus stadtteilgebundenen Merkmalen nicht ableiten. Zurueck kommen der Macro-F1 dieser Zuweisung, der Anteil der so korrekt getroffenen Zeilen und die Verteilung der Modalklassen ueber die Stadtteile.

`marge(panel)`

Zeile 165 · 17 Zeilen

Wie knapp faellt der argmax aus?

Abstand zwischen dem groessten und dem zweitgroessten Klassenanteil je Stadtteil-Monat. Ein kleiner Abstand heisst: das Label haette bei einer anderen Monatsziehung anders gelautet.

ausschoepfung(modelle, basis, a, b)

Zeile 184 · 18 Zeilen

Baselinekorrigierte Quote je Verfahren gegen beide Decken.

(Modell - Mehrheitsklasse) / (Decke - Mehrheitsklasse). Der Rohquotient
Modell/Decke waere geschoent, weil der Sockel der Mehrheitsklasse keine
Leistung des Modells ist.

bericht(tab, aus, mrg, kipp, treffer, modal, n_stadtteile)

Zeile 204 · 38 Zeilen

kein Docstring

main(argv)

Zeile 244 · 66 Zeilen

kein Docstring